Concise Papers

A Comparison of Function Point Counting Techniques

D. R. Jeffery, G. C. Low, and M. Barnes

Abstract—**Effective management of the software development process requires that management be able to estimate total development effort and cost. One of the fundamental problems associated with effort and cost estimation is the *a priori* estimation of software size. Function point analysis has emerged over the last decade as a popular tool for this task. Following its use over this time, however, a number of criticisms of the method have emerged. These criticisms relate to the way in which function counts are calculated and the impact of the processing complexity adjustment on the function point count. SPQR/20 function points among others are claimed to overcome some of these criticisms. This paper compares the SPQR/20 function point method to traditional function point analysis as a measure of software size in an empirical study of MIS environments. In a study of 64 projects in one organization it was found that both methods would appear equally satisfactory. However consistent use of one method should occur since the individual counts differ considerably.**

Index Terms—**Estimation, function points, quantitative measurement, software size.**

I. Introduction

Albrecht [1], [2] introduced the concept of function points as a software size measure while considering the more general problem of measuring application development productivity. His objective was to develop a software size measure that was independent of the implementation technology. To measure productivity, a measure of work product (or output) must be defined. For this Albrecht chose *the function value to be delivered to the user*. To calculate the function value delivered to the user, the number of inputs, outputs, inquiries, and files (including interfaces to other programs) from the user perspective are counted, weighted, and summed. This function *count* is then adjusted for the processing complexity of the application to produce a function *point* number.

Albrecht and Gaffney [3] later published a revision of the function point technique. The revision was based on 18 Cobol and 6 PL/I projects undertaken within IBM DP Services.

A review of the literature reveals five criticisms specifically related to the components and computation of function counts.

- The calculation of function counts tends to take a black box view of the system [4], [16].
- The user defined function types currently established may not be wholly appropriate for current technology [16].
- The classification of the user function types into simple, average, and complex appears to be oversimplified [15].
- The choice of weights was determined by debate and trial [15].
- Function counts are not a technology independent measure of size [15], [16].

Three broad criticisms related to the processing complexity adjustment have also been noted in the literature.

- The restriction to 14 processing complexity factors is not going to be satisfactory for all time, and hence a more open ended approach would seem desirable [15].
- Fourteen processing complexity factors are not necessary for processing complexity to be adequately incorporated into the function point metric [9]
- The application of the processing complexity factor to the raw function point measure contributes little to the performance of function points in the cost estimation process [11]

Manuscript received June 3, 1991; revised May 5, 1992; Recommended by M. Deutsch.

The authors are with the School of Information Systems, University of New South Wales, Kensington 2033 NSW, Australia.

IEEE Log Number 9208767.

A. Modifying Function Point Analysis

A number of modifications to function point analysis (FPA) have been suggested. Jones [9] in the SPQR estimating tool implements a variation (herein designated as SPQR/20 fp).

The SPQR/20 fp metric addresses two criticisms applied to FPA:

1) the classification of user defined functions into simple, average, and complex; and
2) the use of 14 complexity adjustment factors.

Within SPQR, all user defined functions are given a single classification. The weights applied to the single classification are the same as those applied for average complexity in FPA. This represents a partial return to the initial FPA counting technique of 1979 [1] and addresses Jones's [9] concern that the three classifications are not needed. However it does not address the concerns of either Symons [15] or Verner *et al.* [16] that more, not fewer, classifications are needed.

In addition in SPQR/20 fp there are only two complexity adjustment factors; logical complexity and data complexity. These two are rated on a five point scale and can adjust the raw SPQR/20 fp count by ±40%.

Jones [9] claims that this method of function point analysis produces function point totals that are essentially the same as FPA. Averaged over a number of projects this is not totally unexpected since the SPQR/20 fp method uses the aveage weights that are used in FPA. However Jones [9] also asserts that the individual counts are generally within 1% of one another.

Thus the argument proposed in favor of SPQR/20 fp lies with its relative counting simplicity. It therefore should be easier and less costly to count.

The aim of the research was to test the claims that the simplified SPQR/20 fp counting technique would result in a size measure virtually the same as traditional FPA. This was done by collecting data on 64 systems developed by one organization (see [13]).

II. Research Objectives and Data

A. Research Objective

The aim of testing for differences between FPA and SPQR/20 fp was initially broken into three hypotheses.

H1—Function Counts: In an MIS environment, there is no significant difference between the ex post function counts produced by the FPA and SPQR/20 fp method.

H2—Complexity Adjustment Factor: In an MIS environment, there is no significant difference between the processing complexity adjustments calculated for the FPA and SPQR/20 fp methods.

H3—Function Points: In an MIS environment, there is no significant difference between the ex post function points produced by the FPA and SPQR/20 fp methods.

0098-5589/93$03.00 © 1993 IEEE

Authorized licensed use limited to: California Polytechnic State University San Luis Obispo. Downloaded on March 11,2025 at 08:10:43 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PROJECT SIZE AND DEVELOPMENT EFFORT DATA

No. of Projects	Project Size (FPA)			Development Effort (days)		
	Mean	Std. Dev.	Range	Mean	Std. Dev.	Range
64	189.0	203.2	25–1184	265.5	257.5	21–1182

In addition, the following two hypotheses were proposed to test whether the FPA and SPQR/20 fp methods result in effort/size equations of different accuracy.

H4—Existence of an Effort Relationship: In an MIS environment, there is no relationship between actual development effort and either the ex post function counts or function points produced by each of the FPA and SPQR/20 fp methods.

H5—Accuracy of the Effort Relationship: In an MIS environment, there is no difference in the accuracy of the derived effort estimating equation when the independent variable, software size, is measured using either ex post function counts or function points produced by each of the FPA and SPQR/20 fp methods.

B. Data Collection

Data were collected from systems in a large public utility which had been using function points for over five years. The organization was relatively mature in terms of its use of function points for productivity analysis, and had developed a centralized database for the recording of function points and related measurement data. It used these data as the basis for estimating required effort on development projects.

Data were collected on:

1) project size measured using FPA;
2) effort expended on the project; and
3) factors to enable the calculation of SPQR/20 fp.

Data were available for a total of 64 projects of which 3 were developed in Cobol and 61 were developed in PL/I. The projects were for both batch and on-line environments. Some utilized available database software (SQL), while others had used back-end tools for code generation.

The function point counting technique in the organization was reviewed to ensure that FPA counting conventions were in use.

Effort data for each project were measured by the organization in days. The data are summarized in Table I.

SPQR/20 fp data were derived directly from the collected FPA data for each project. Function points were converted into SPQR/20 fp in two steps.

1) The number of each user defined function type was reweighted according to the SPQR/20 model.
2) The two processing complexity factors, logical complexity and data complexity, were calculated based on examination of the fourteen processing complexity characteristics available with each FPA function point count.

When calculating the SPQR/20 fp complexity adjustment factors, the following approach was adopted by the authors:

- logical complexity was derived from an average of the data communications, on-line data entry, complex processing logic, and desire to facilitate change processing complexity characteristics. For example, if the FPA complexity adjustments for data communications, on-line data entry, complex processing logic, and desire to facilitate change were 3, 2, 1, and 2, respectively, then the SPQR/20 logical complexity would be 2 (i.e., [3 + 2 + 1 + 2]/4); and
- Data complexity was derived from an average of the distributed processing, and on-line update processing complexity characteristics. For example, if the distributed processing complexity was 3 and the on-line update processing complexity was 1, then the SPQR/20 data complexity would be 2 (i.e., [3 + 1]/2).

The other processing complexity factors of performance, operational configuration load, transaction rate, end user efficiency, code reusability, installation ease, operational ease, and multiple sites were not included in the derivation as they appeared to have no impact upon the processing complexity factors as defined in the SPQR/20 fp model. This approach was adopted following an examination of project specifications and discussions with four project leaders within the organization.

A summary of the SPQR/20 fp data is given in Table II.

TABLE II
SPQR/20 fp DATA

No. of Projects	SPQR/20 fp		
	Mean	Std. Dev.	Range
64	179.0	191.2	20–862

TABLE III
KOLMOGOROV–SMIRNOV GOODNESS OF FIT

No. of Projects	Kolmogorov–Smirnov Goodness of Fit			
	FPA		SPQR/20 fp	
	Function Count p	Function Point p	Function Count p	Function Point p
64	0.004	0.007	0.006	0.010

TABLE IV
FPA AND SPQR/20 FUNCTION COUNTS

No. of Projects	FPA Function Counts			SPQR/20 Function Counts		
	Mean	Std. Dev.	Range	Mean	Std. Dev.	Range
64	204.1	223.9	32–1393	232.7	239.1	43–1289

III. DATA ANALYSIS AND DISCUSSION

The Kolmogorov–Smirnov test indicates that the function count and function point data belong to normal distributions [10] (see Table III).

A. H1—Function Counts

The FPA and SPQR/20 function counts are presented in Table IV. The FPA function count is approximately 14% lower on average than the SPQR/20 function count. Since the SPQR/20 fp method assumes an average complexity for the user function types, this implies that the organization studied when using FPA weighted its function types below average.

A 2-tailed paired t-test was used to determine if there was a significant difference between the ex post function counts produced by FPA and SPQR/20 fp analysis. The results ($t = -4.80, p = 0.000$) indicate that there is a significant difference between the FPA and SPQR/20 function counts.

B. H2—Complexity Adjustment Factor

The mean complexity adjustment factors are shown in Table V. The variation in the complexity adjustment factor is to be expected since the projects studied included both batch and on-line projects, projects using embedded database commands, and code generated using back-end CASE tools.

Authorized licensed use limited to: California Polytechnic State University San Luis Obispo. Downloaded on March 11,2025 at 08:10:43 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PROCESSING COMPLEXITY ADJUSTMENTS: DESCRIPTIVE STATISTICS

No. of Projects	Complexity Adj. for FPA Function Points			Complexity Adj. for SPQR/20 Function Points		
	Mean	Std. Dev.	Range	Mean	Std. Dev.	Range
64	0.909	0.089	0.72–1.11	0.723	0.185	0.45–1.17

TABLE VI
FPA AND SPQR/20 FUNCTION POINTS

No. of Projects	FPA Function Points			SPQR/20 Function Points		
	Mean	Std. Dev.	Range	Mean	Std. Dev.	Range
64	189.0	203.2	25–1184	179.0	191.2	20–862

A 2-tailed paired t-test was used to test hypothesis H2 that in a MIS environment there is no statistical difference between the complexity adjustment factors for FPA and SPQR/20 fp analysis. The results ($t = 12.08, p = 0.000$) indicate that the complexity adjustment factors for FPA and SPQR/20 fp analysis are significantly different.

C. H3—Function Points

The results for hypotheses H1 and H2 indicate that the function counts and the complexity adjustment factors are significantly different for FPA and SPQR/20 fp analysis. Function points are calculated from the function count and the complexity adjustment factor. The FPA and SPQR/20 function points are presented in Table VI.

A 2-tailed paired t-test was initially used to determine if the ex-post function points produced by FPA and SPQR/20 fp analysis are significantly different. The results ($t = 1.19, p = 0.239$) show that there is no significant difference between the FPA and SPQR/20 function point means. This is somewhat surprising given that there is a significant difference between the function counts (H1) and the complexity adjustment factors (H2) for FPA and SPQR/20 fp analysis. In this case, a comparison of variances may be used to assess the variability of the individual results for FPA and SPQR/20 fp even though a paired t-test suggests no significant difference. The Hartley test is recommended in this case "because of its superior power and ease of computation" provided the underlying treatment populations are normal [12]. The Hartley test allows a statistical comparison of within-group variances. Since the application of the complexity adjustment factor has moved the means toward each other, the interesting question is whether the variances of the two distributions are significantly different.

Since the Kolmogorov–Smirnov test in this case indicates that both the FPA and SPQR/20 function point data belong to a normal distribution ($p = 0.007$ and $p = 0.010$, respectively) it is reasonable to use the Hartley test. The Hartley test causes rejection of the hypothesis of homogeneity of variance ($p > 0.05$).

Hence FPA and SPQR/20 function points cannot be used interchangeably because of the significantly different variance in the population.

To further illustrate this point, the absolute relative error (ARE) was computed for each project, and the mean absolute relative error (MARE) for the organization. The ARE and MARE are calculated as [5]:

$$\text{ARE} = \left| \frac{\text{SPQR/20 fp} - \text{FPA function points}}{\text{FPA function points}} \right|$$

$$\text{MARE} = \frac{1}{n} \sum_{i=1} \text{ARE}.$$

TABLE VII
ARE FREQUENCIES—SPQR/20 fp COMPARED TO FPA FUNCTION POINTS

% change FPA versus SPQR/20 fp	Number of Projects
<= 1%	4
<= 2%	2
<= 5%	2
<= 10%	9
<= 20%	21
<= 40%	22
> 40%	4
Overall	64

TABLE VIII
EFFORT EQUATION REGRESSION RESULTS FOR FUNCTION COUNTS AND FUNCTION POINTS

Method	No. of Projects	Function Counts		Function Points	
		r^2	p	r^2	p
FPA	64	0.364	0.0000	0.372	0.0000
SPQR/20	64	0.340	0.0000	0.335	0.0000

The mean MARE was 0.193 with standard deviation of 0.126. To ensure that the MARE calculations was not unduly affected by outlier data points, Table VII is presented.

Given that only 4 of a possible 64 data points fall within the 1% range and the Hartley test results, Jones' [9] assertion that in most cases FPA and SPQR/20 function points do not differ by more than one percent must be rejected.

D. H4—Existence of an Effort Relationship

The fourth hypothesis investigates the existence of a relationship between actual development effort and the ex post function points produced by either FPA or SPQR/20 fp analysis. This hypothesis ws investigated using simple regression analysis.

Various authors [4], [5], [7], [8] have suggested that a nonlinear relationship is appropriate between development effort and software size, when measured using lines of code:

$$\text{effort} = a^*(\text{lines of code})^b * f(\text{other factors}) \quad (1)$$

where a and b are constants.

Expressing the above equation using function points as the software size measure only changes the leading constant [14]. Hence,

$$\text{effort} = c^*(\text{function points})^b * f(\text{other factors}) \quad (2)$$

where c and b are constants.

This form of effort equation has been suggested by IBM [6] and used by Low and Jeffery [14].

Regression using actual development effort as the dependent variable and ex post function counts or function points (FPA or SPQR/20) as the independent variable, yielded the results in Table VIII.

Both r^2's were significant at the 99% level of confidence. The results indicate a relationship between actual development effort and software size when measured by either FPA or SPQR/20 fp (functio counts or function points).

Less than 40% of the variation in actual development effort is explained by the function point measures. This suggests that:

- the effort variance in this organization is high; and/or

Authorized licensed use limited to: California Polytechnic State University San Luis Obispo. Downloaded on March 11,2025 at 08:10:43 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
EFFORT ESTIMATE ARE t-TESTS FOR FPA AND SPQR/20 fp

No. of Projects	FPA		SPQR/20 fp		t	p
	Mean ARE	Std. Dev.	Mean ARE	Std. Dev.		
64	0.1193	0.101	0.1226	0.103	−0.95	0.344

TABLE X
EFFORT ESTIMATE ARE t-TESTS FOR FUNCTION COUNTS AND FUNCTION POINTS

Method	No. of Projects	Function Count		Function Point		t	p
		Mean ARE	Std. Dev.	Mean ARE	Std. Dev.		
FPA	64	0.1193	0.104	0.1193	0.010	0.02	0.983
SPQR/20	64	0.1228	0.104	0.1226	0.103	0.06	0.950

- the remaining (60%+) variation can be explained by factors (i.e., f(other factors) in (1) and (2) such as staff experience) not accounted for by either of the function point metrics.

The conclusion is that there is little difference between FPA and SPQR/20 fp (function count or function point) for the purpose of determining an effort relationship.

E. H5—Accuracy of the Effort Relationship

This hypothesis concerns the difference in the accuracy of the effort equations derived from the regression analysis if the independent variable, software size, is measured using ex post function counts or function points produced by either FPA or SPQR/20 fp analysis.

To test whether using different function point metrics as the independent variable results in effort equations of differing accuracy, a two-tailed paired t-test was used to compare the absolute relative errors (ARE) in the effort estimate for each individual project.

The results in Table IX indicate that there was no statistically significant difference ($p > 0.05$) in the accuracy of the derived effort equations using either FPA or SPQR/20 fp.

To test whether the function count or function point measures resulted in effort equations of differing accuracy, a two-tailed paired t-test was used to compare the absolute relative errors (ARE) of the effort estimates. The results in Table X seem to indicate that in both the FPA and the SPQR/20 fp method the processing complexity adjustment does not affect the accuracy of the derived effort equations. This conclusion validates the comments of Kemerer [11] and Symons [15] relating to the usefulness of the 14 processing complexity factors. While the 14 processing complexity factors did not provide useful input to the FPA method, neither does the SPQR/20 complexity factor. This latter fact is contrary to the assertion of Jones [9].

IV. CONCLUSIONS

In recent years, there has been a growing demand that the software development process be effectively managed, producing cost effective, reliable software within specified time constraints. Critical to the effective management of a software development is the ability of management to forecast the total development effort and cost. One of the fundamental problems associated with effort and cost estimation is determining accurate estimates of software size. Function point analysis has recently emerged as a popular tool for determining this.

Following a decade of use, the function point method has been subject to a number of criticisms, relating to both the way in which function counts are calculated, and the precise impact of the processing complexity adjustment when applied to the raw function point count. Jones [9] suggested SPQR/20 fp method addresses some of these criticisms. This research examined

- the extent that the function count differs using the FPA and SPQR/20 fp analysis; and
- difference in the accuracy of the effort equations using either FPA function points or SPQR/20 fp.

Three broad conclusions are suggested.

1) As an ex post measure of software size, both the FPA and the SPQR/20 fp method would appear equally satisfactory in an MIS environment. However consistent use of the one method should occur since the individual counts can differ considerably.
2) In FPA and SPQR/20 fp analysis, the processing complexity adjustment does not appear to affect the accuracy of the derived effort equations. This result suggests that a review of the role of the processing complexity adjustment may be appropriate at this time.
3) There exists a relationship between actual development effort and both the ex post function counts and function points produced by FPA and SPQR/20 fp analysis. The result related to FPA further supports past research in this area [3], [11].

Why the processing complexity adjustment had no effect on the accuracy of the resulting effort equation needs to be determined. This necessitates researching the way in which the processing complexity adjustment is used by function point counters, whether other factors should be included in the processing complexity adjustment, or whether the processing complexity adjustment should be eliminated altogether and perhaps other environmental variables substituted.

REFERENCES

[1] A. J. Albrecht, "Measuring application development productivity," in *Proc. Joint SHARE/GUIDE/IBM Application Development Symp.,* Oct. 1979, pp. 83–92.

[2] ——, *AD/M Estimating and Productivity Measurement Guidelines,* IBM Corp. Information Systems, 1984.

[3] A. J. Albrecht and J. E. Gaffney, "Software function, source lines of code and development effort prediction: A software science validation," *IEEE Trans. Software Eng.,* vol. SE-9, no. 6, pp. 639–648, Nov. 1983.

[4] B. W. Boehm, *Software Engineering Economics.* Englewood Cliffs, NJ: Prentice-Hall, 1981, pp. 316–317.

[5] S. D. Conte, H. E. Dunsmore, and V. Y. Shen, *Software Engineering Metrics and Models.* Menlo Park, CA: Benjamin/Cummings, CA: 1983.

[6] *Development Centre Productivity Guide,* IBM, Middlesex, England, 1984.

[7] D. R. Jeffery, "The relationship between team size, experience and attitudes and software development productivity," in *Proc. IEEE COMPSAC 87,* Tokyo, Japan, Oct. 1987, pp. 2–8.

[8] D. R. Jeffery and G. C. Low, "Calibrating estimation tools for software development," *Software Eng. J.,* vol. 5, no. 4, pp. 215–221, July 1990.

[9] T. C. Jones, "A short history of function points and feature points," Software Productivity Reserch Inc., 1988.

[10] C. H. Hull and N. H. Nie, *SPSS Update 7- 9,* New York: McGraw-Hill, 1981, p. 224.

[11] C. F. Kemerer, "An empirical validation of software cost estimation models," *Commun. ACM,* pp. 416–429, May 1987.

[12] G. Keppell, *Design and Analysis: A Researchers Handbook.* Englewood Cliffs, NJ: Prentice-Hall, 1982, pp. 96–99.

[13] G. C. Low and D. R. Jeffery, "Function points in the estimation and evaluation of the software process," *IEEE Trans. Software Eng.,* vol. 16, no. 1, pp. 64–71, Jan. 1990.

[14] ——, "Productivity issues in the use of current back-end CASE tools," in *Proc. 3rd Int. Workshop CASE, CASE 89,* IEEE and BCS, London, England, July 1989, pp. 12–38.

[15] C. R. Symons, "Function point analysis: Difficulties and improvements," *IEEE Trans. Software Eng.,* vol. 14, no. 1, pp. 2–11, 1988.

[16] J. Verner, G. Tate, B. Jackson, and R. Hayward, "Technology dependence in function point analysis: A case study and critical review," Dep. Comput. Sci., Massey Univ. Sept., 1988.

Authorized licensed use limited to: California Polytechnic State University San Luis Obispo. Downloaded on March 11,2025 at 08:10:43 UTC from IEEE Xplore. Restrictions apply.